EECE 5639: Computer Vision
Fall 2026, Homework 2

Jingqi Mei

October 9, 2026

Problem 1

$I_k = \hat{I}_k + N_k, \quad N_k \sim \mathcal{N}(0, \sigma^2)$ i.i.d., $\quad I_i^{\text{out}} = w_1 I_{i-1} + w_2 I_i + w_3 I_{i+1}, \quad w_1 + w_2 + w_3 = 1.$

(a)

$$I_i^{\text{out}} = \left(w_1 \hat{I}_{i-1} + w_2 \hat{I}_i + w_3 \hat{I}_{i+1}\right) + (w_1 N_{i-1} + w_2 N_i + w_3 N_{i+1})$$

$$\boxed{N_i^{\text{out}} = w_1 N_{i-1} + w_2 N_i + w_3 N_{i+1}}$$

(b)

The noise has zero mean and is independent, so all cross terms $E[N_j N_k]$, $j \neq k$, vanish:

$$\boxed{\text{Var}(N_i^{\text{out}}) = \sigma^2 \left(w_1^2 + w_2^2 + w_3^2\right)}$$

(c)

$$L(w_1, w_2, w_3; \lambda) = \sigma^2 \left(w_1^2 + w_2^2 + w_3^2\right) + \lambda \left(w_1 + w_2 + w_3 - 1\right)$$

$$\frac{\partial L}{\partial w_k} = 2\sigma^2 w_k + \lambda = 0 \;\Rightarrow\; w_k = -\frac{\lambda}{2\sigma^2} \quad (k = 1, 2, 3), \qquad \frac{\partial L}{\partial \lambda} = w_1 + w_2 + w_3 - 1 = 0.$$

Hence $w_1 = w_2 = w_3$, and the constraint gives $3\left(-\frac{\lambda}{2\sigma^2}\right) = 1$, so $\lambda^* = -\frac{2\sigma^2}{3}$ and

$$\boxed{w_1^* = w_2^* = w_3^* = \tfrac{1}{3}}$$

This is the minimum: the objective is strictly convex (Hessian $2\sigma^2 \mathbf{I} \succ 0$) and the constraint is linear.

(d)

The **box (mean) filter** $\left[\frac{1}{3}\ \frac{1}{3}\ \frac{1}{3}\right]$.

(e)

$\sigma_{\text{out}}^2 = 3 \cdot \frac{1}{9}\sigma^2 = \sigma^2/3$, so the noise variance is reduced by a **factor of 3**.

(f)

The box weights all neighbours equally, so pixels on the other side of an edge count as much as the centre pixel. Edges become blurred ramps, and thin lines and fine details lose contrast. Its sinc-shaped frequency response also has sidelobes, which cause ringing and blocky artifacts. A Gaussian puts most of the weight on the nearest pixels, so it keeps edges and details sharper. It has a smooth frequency response with no ringing, and its smoothing strength can be tuned with σ. It removes slightly less noise than the box but usually gives a better-looking image.

Problem 2

Code: `hw2_q2.py` (Appendix). The image is normalized to $[0, 1]$. Panel titles show the PSNR with respect to the clean image.

(a)

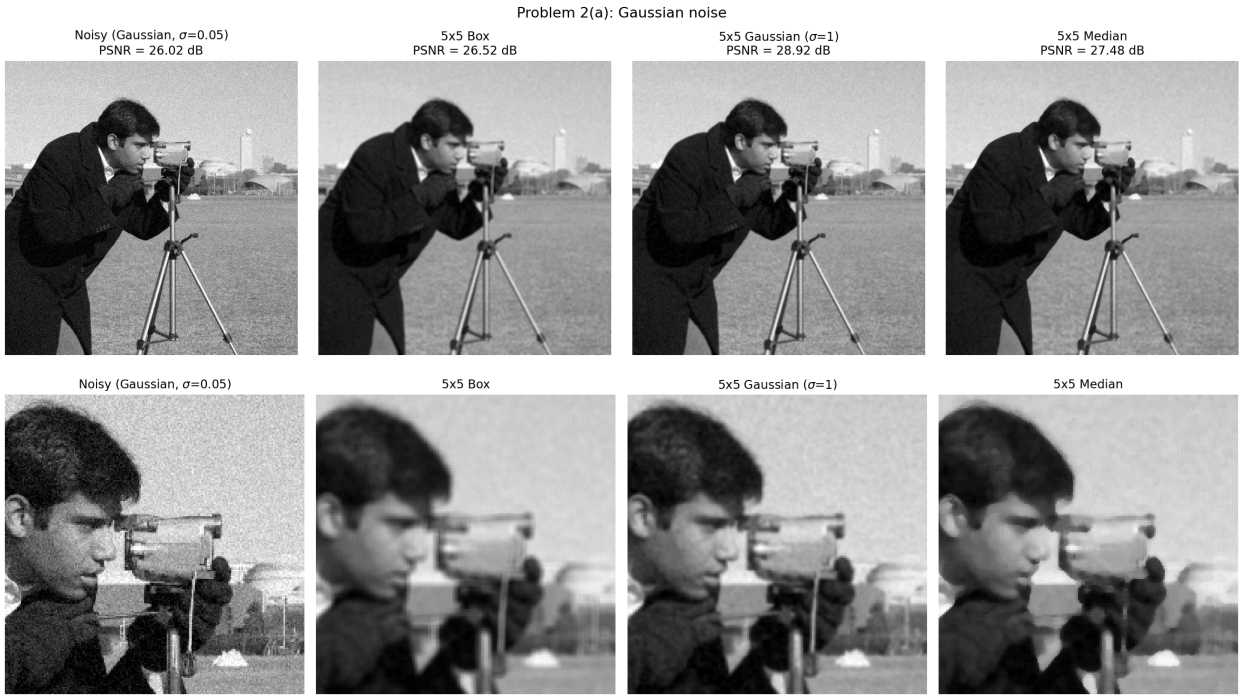


Figure 1: Gaussian noise ($\sigma = 0.05$) and the three 5×5 filters (bottom: zoom).

	Noisy	Box	Gaussian	Median
PSNR (dB)	26.02	26.52	**28.92**	27.48
Residual noise std (flat sky)	0.0499	**0.0099**	0.0142	0.0126
Edge gradient (clean: 1.04)	1.043	0.442	**0.656**	0.586

(b)

i. The **Box filter** removes the most noise. It leaves the lowest residual noise (0.0099), which matches the theoretical 0.05/5.

ii. **No.** The box image is the blurriest: edges, the face and the tripod are smeared, and it has the lowest PSNR. The **Gaussian** gives the best-looking image. The median keeps strong edges but makes flat areas look blotchy and removes thin details.

iii. The box gives all 25 pixels the same weight (0.04). The Gaussian weights the centre most (0.16) and the corners very little (0.003). The box therefore averages more, removing more noise but causing more blur. The Gaussian averages more locally and keeps more detail.

(c)

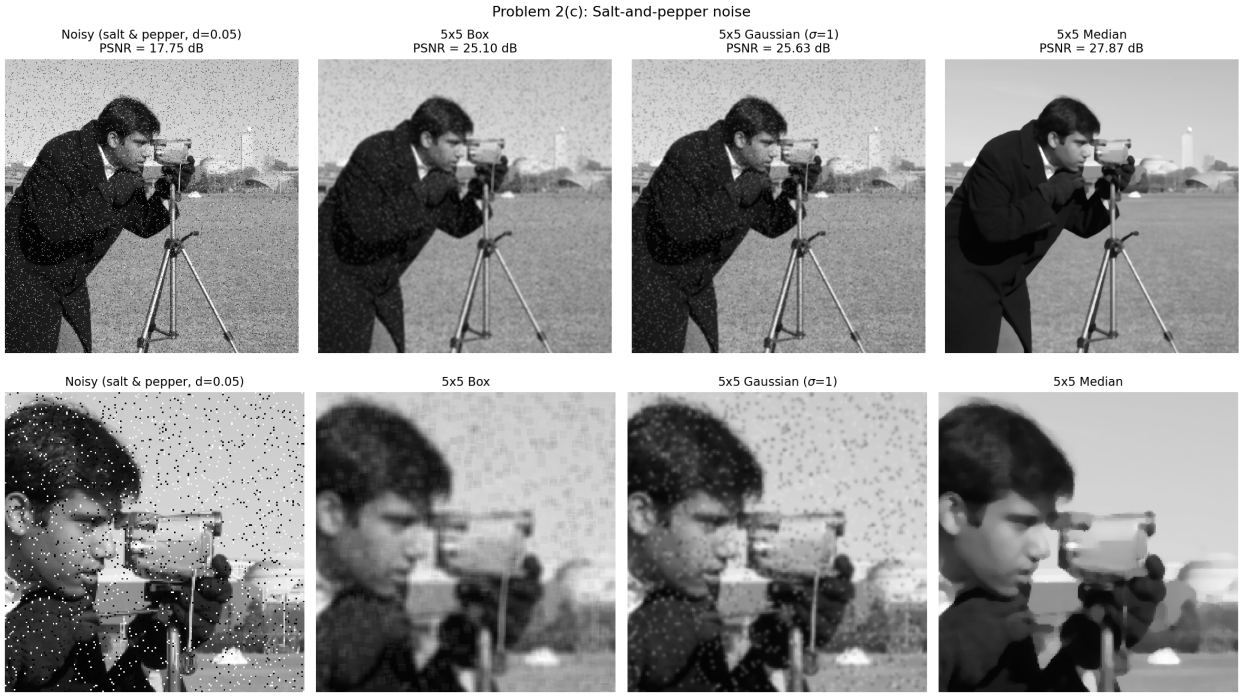


Figure 2: Salt-and-pepper noise ($d = 0.05$) and the three 5×5 filters (bottom: zoom).

(d)

i. The **Median filter**: almost all impulses are removed (27.87 dB vs. about 25 dB for Box and Gaussian).

ii. Salt-and-pepper pixels are outliers. A 5×5 window contains only about 1–2 of them, so the median ignores them and outputs the value of a real neighbouring pixel. Linear filters instead spread each impulse into a visible blob.

iii. In the Gaussian-noise case the median was only second (27.48 dB vs. 28.92 dB for the Gaussian). There every pixel is slightly corrupted and there are no outliers to reject, and the median gives blotchy results. It works best for impulse noise.

(e)

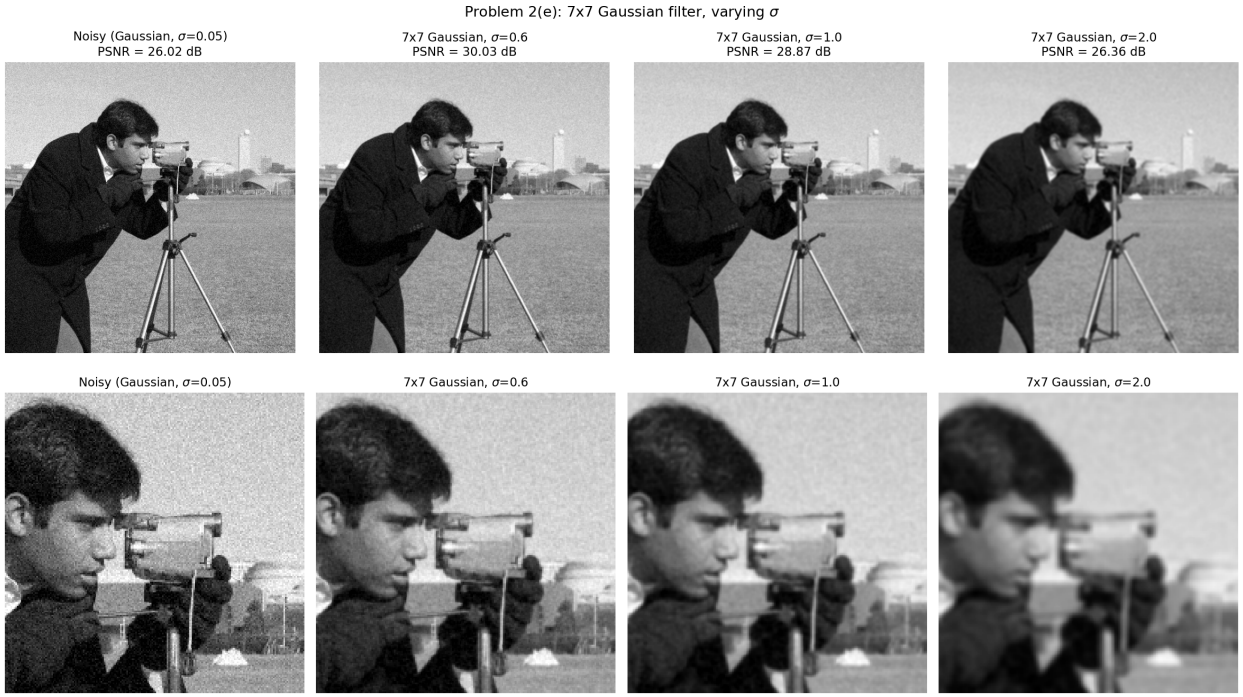


Figure 3: 7×7 Gaussian filter with $\sigma = 0.6, 1, 2$ (bottom: zoom).

	$\sigma = 0.6$	$\sigma = 1$	$\sigma = 2$
1-D centre weight	0.664	0.399	0.216
Residual noise std (flat sky)	0.0246	0.0140	**0.0081**
Edge gradient (clean: 1.04)	**0.850**	0.647	0.400
PSNR (dB)	**30.03**	28.87	26.36

(f)

As σ increases:

- **Noise reduction increases**: the residual noise drops from 0.0246 to 0.0081.
- **More fine detail is lost**: the hair, the face and the grass texture fade away at $\sigma = 2$.
- **Edges become less sharp**: the edge gradient drops from 0.85 to 0.40.

This is consistent with the kernel weights. A small σ concentrates the weight on the centre pixel (0.66), so there is little averaging. A large σ spreads the weight almost evenly over the 7×7 window, so the filter behaves like a box: more averaging, less noise, more blur.

(g)

1. **Noise type:** for Gaussian noise, which affects every pixel, use a linear filter; the Gaussian is best. For salt-and-pepper noise, which consists of outliers, use the median filter.

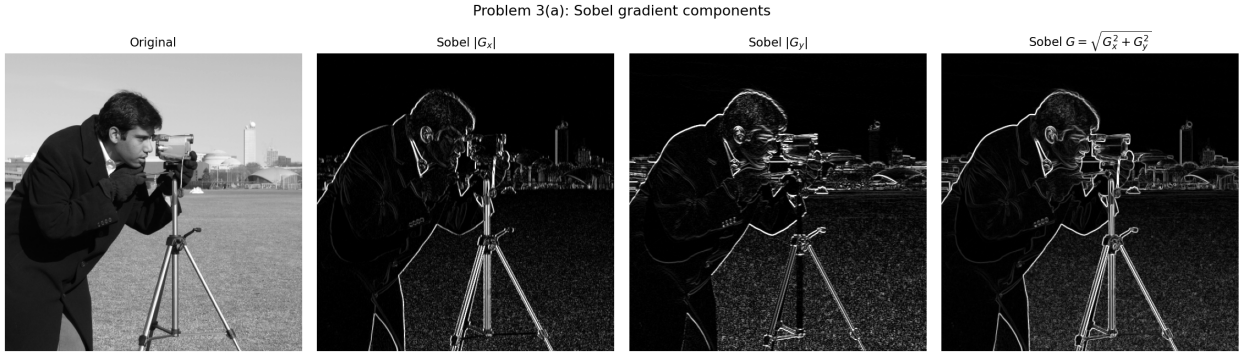
2. **Trade-off:** stronger smoothing (a box, or a Gaussian with large σ) removes more noise but blurs edges and details. The filter that removes the most noise does not give the best image.

3. **Choosing σ:** use the smallest σ that removes enough noise. Here $\sigma \approx 0.6$–1 works well and $\sigma = 2$ over-smooths.

Problem 3

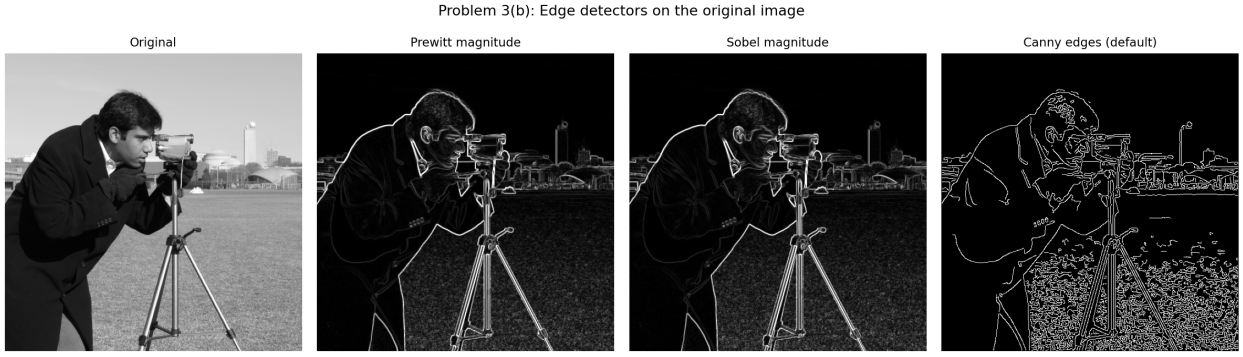
Code: `hw2_q3.py` (Appendix). Prewitt and Sobel come from `scipy.ndimage`; Canny is `skimage.feature.canny` with default settings. Gradient images are scaled to their 99.5th percentile for display.

(a)



$|G_x|$ responds to horizontal intensity changes, so it highlights **vertical edges** (the coat outline, the tripod legs, the buildings). $|G_y|$ highlights **horizontal edges** (the horizon, the top and bottom of the camera, the shoulders). G combines the two, showing edges of all orientations.

(b)

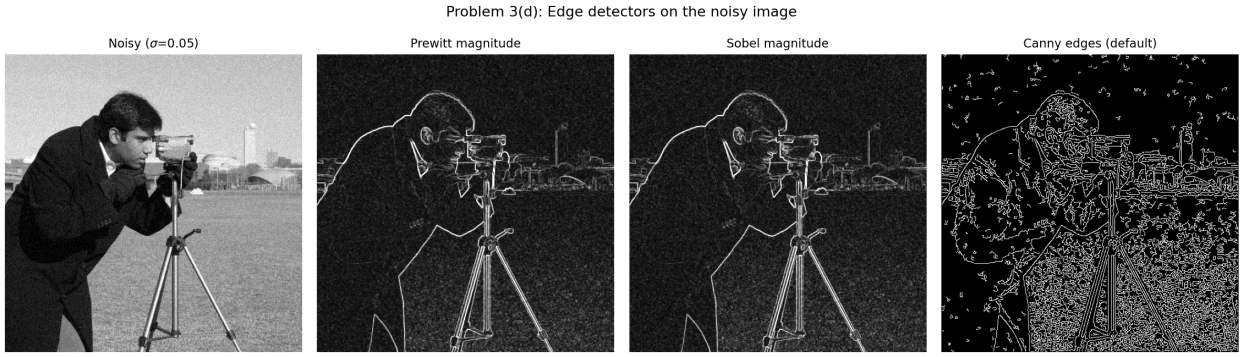


(c)

- **Boundaries:** all three methods find the main boundaries (the man, the camera, the tripod, the skyline). In Prewitt and Sobel, low-contrast edges appear faint. Canny marks every detected edge equally.

- **Thickness and continuity:** Prewitt and Sobel edges are thick (2–3 pixels) and vary in brightness. Canny edges are thin (1 pixel) and mostly continuous because of non-maximum suppression and hysteresis.
- **Differences:** Prewitt and Sobel look almost identical. Canny gives a binary edge map, but with the default thresholds it also marks a lot of grass texture.

(d)



	Clean	Noisy
Prewitt: sky response / edge response	0.0047	0.0922
Sobel: sky response / edge response	0.0048	0.0948
Canny: edge pixels in 100×100 sky patch	0	218
Canny: clean edges still detected	–	94%

(e)

i. **Most sensitive: Prewitt and Sobel** (about equally). The whole background becomes grainy, and the noise level relative to the edges rises about 20×.

ii. **Most robust: Canny.** The main contours stay clean and connected (94% preserved), and only a few short noise fragments appear.

iii. Derivatives amplify high-frequency noise, so the amount of smoothing matters:

- *Prewitt*: a difference combined with a 3-pixel average. Very little smoothing.
- *Sobel*: a difference combined with $[1, 2, 1]$ smoothing. The window is just as small, so it is about equally noisy.
- *Canny*: Gaussian smoothing ($\sigma = 1$) *before* computing gradients, which greatly reduces the noise. Non-maximum suppression and hysteresis thresholding then remove most of the isolated noise responses.

Appendix: Code

`hw2_q2.py`

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage
from skimage import io

rng = np.random.default_rng(5639)


def load_image(path="cameraman.tif"):
    img = io.imread(path).astype(np.float64)
    return (img - img.min()) / (img.max() - img.min())


def gaussian_kernel(size, sigma):
    r = size // 2
    x = np.arange(-r, r + 1)
    g = np.exp(-x**2 / (2 * sigma**2))
    k = np.outer(g, g)
    return k / k.sum()


def box_filter(img, size=5):
    return ndimage.uniform_filter(img, size=size, mode="reflect")


def gaussian_filter(img, size=5, sigma=1.0):
    return ndimage.convolve(img, gaussian_kernel(size, sigma), mode="reflect")


def median_filter(img, size=5):
    return ndimage.median_filter(img, size=size, mode="reflect")


def add_gaussian_noise(img, sigma=0.05):
    return img + rng.normal(0.0, sigma, img.shape)


def add_salt_pepper(img, density=0.05):
    noisy = img.copy()
    u = rng.random(img.shape)
    noisy[u < density / 2] = 0.0
    noisy[(u >= density / 2) & (u < density)] = 1.0
    return noisy


def psnr(ref, img):
    mse = np.mean((ref - img) ** 2)
    return 10 * np.log10(1.0 / mse)


def show_row(images, titles, ref, fname, suptitle):
    fig, axes = plt.subplots(1, len(images), figsize=(4 * len(images), 4.4))
    for ax, im, t in zip(axes, images, titles):
        ax.imshow(np.clip(im, 0, 1), cmap="gray", vmin=0, vmax=1)
        ax.set_title(f"{t}\nPSNR = {psnr(ref, im):.2f} dB", fontsize=11)
        ax.axis("off")
    fig.suptitle(suptitle, fontsize=13, y=1.03)
    fig.tight_layout()
    fig.savefig(f"figures/{fname}", dpi=150, bbox_inches="tight")
    plt.close(fig)


def show_zoom(images, titles, fname, rows=slice(60, 260), cols=slice(180, 380)):
    fig, axes = plt.subplots(1, len(images), figsize=(4 * len(images), 4.2))
    for ax, im, t in zip(axes, images, titles):
        ax.imshow(np.clip(im[rows, cols], 0, 1), cmap="gray", vmin=0, vmax=1,
                  interpolation="nearest")
```

```
        ax.set_title(t, fontsize=11)
        ax.axis("off")
    fig.tight_layout()
    fig.savefig(f"figures/{fname}", dpi=150, bbox_inches="tight")
    plt.close(fig)


def main():
    I = load_image()

    In = add_gaussian_noise(I, 0.05)
    res_a = [In, box_filter(In), gaussian_filter(In, 5, 1.0), median_filter(In)]
    titles = ["Noisy (Gaussian, $\\sigma$=0.05)", "5x5 Box", "5x5 Gaussian ($\\sigma$=1)", "5x5 Median"]
    show_row(res_a, titles, I, "q2a_gaussian_noise.png", "Problem 2(a): Gaussian noise")
    show_zoom(res_a, titles, "q2a_zoom.png")

    Isp = add_salt_pepper(I, 0.05)
    print(f"(c) fraction of corrupted pixels: {np.mean(Isp != I):.4f}")
    res_c = [Isp, box_filter(Isp), gaussian_filter(Isp, 5, 1.0), median_filter(Isp)]
    titles = ["Noisy (salt & pepper, d=0.05)", "5x5 Box", "5x5 Gaussian ($\\sigma$=1)", "5x5 Median"]
    show_row(res_c, titles, I, "q2c_salt_pepper.png", "Problem 2(c): Salt-and-pepper noise")
    show_zoom(res_c, titles, "q2c_zoom.png")

    sigmas = [0.6, 1.0, 2.0]
    res_e = [In] + [gaussian_filter(In, 7, s) for s in sigmas]
    titles = ["Noisy (Gaussian, $\\sigma$=0.05)"] + [f"7x7 Gaussian, $\\sigma$={s}" for s in sigmas]
    show_row(res_e, titles, I, "q2e_gaussian_sigma.png", "Problem 2(e): 7x7 Gaussian filter, varying $\\sigma$"
        )
    show_zoom(res_e, titles, "q2e_zoom.png")

    print("\nPSNR (dB) w.r.t. clean image")
    for name, res in [("(a) Gaussian noise", res_a), ("(c) Salt & pepper", res_c)]:
        print(f" {name}: noisy={psnr(I, res[0]):.2f}, box={psnr(I, res[1]):.2f}, "
              f"gauss={psnr(I, res[2]):.2f}, median={psnr(I, res[3]):.2f}")
    print(" (e) " + ", ".join(f"sigma={s}: {psnr(I, r):.2f}" for s, r in zip(sigmas, res_e[1:])))

    flat = (slice(20, 120), slice(20, 120))
    print("\nResidual std in flat sky patch / mean Sobel gradient magnitude of clean-image edges")
    edge_mask = np.hypot(ndimage.sobel(I, 1), ndimage.sobel(I, 0)) > 0.5
    for name, im in zip(["noisy", "box", "gauss", "median"], res_a):
        g = np.hypot(ndimage.sobel(im, 1), ndimage.sobel(im, 0))
        print(f" (a) {name:6s}: flat std={np.std(im[flat] - I[flat]):.4f}, edge grad={g[edge_mask].mean():.3f}"
            )
    for s, im in zip(sigmas, res_e[1:]):
        g = np.hypot(ndimage.sobel(im, 1), ndimage.sobel(im, 0))
        print(f" (e) sigma={s}: flat std={np.std(im[flat] - I[flat]):.4f}, edge grad={g[edge_mask].mean():.3f}"
            )

    print("\nCentre row of normalized 7x7 Gaussian kernels:")
    for s in sigmas:
        k = gaussian_kernel(7, s)
        print(f" sigma={s}: centre weight={k[3, 3]:.4f}, row = {np.round(k[3] / k[3].sum(), 4)}")


if __name__ == "__main__":
    main()
```

hw2_q3.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage
from skimage import io, feature

rng = np.random.default_rng(5639)

```

```python

def load_image(path="cameraman.tif"):
    img = io.imread(path).astype(np.float64)
    return (img - img.min()) / (img.max() - img.min())


def gradients(img, method):
    op = {"sobel": ndimage.sobel, "prewitt": ndimage.prewitt}[method]
    Gx = op(img, axis=1, mode="reflect")
    Gy = op(img, axis=0, mode="reflect")
    return Gx, Gy, np.sqrt(Gx**2 + Gy**2)


def canny(img):
    return feature.canny(img)


def show_row(images, titles, fname, suptitle):
    fig, axes = plt.subplots(1, len(images), figsize=(4 * len(images), 4.4))
    for ax, im, t in zip(axes, images, titles):
        ax.imshow(im, cmap="gray", vmin=0, vmax=max(im.max(), 1e-12) if im.dtype != bool else 1)
        ax.set_title(t, fontsize=11)
        ax.axis("off")
    fig.suptitle(suptitle, fontsize=13, y=1.03)
    fig.tight_layout()
    fig.savefig(f"figures/{fname}", dpi=150, bbox_inches="tight")
    plt.close(fig)


def scale(g):
    return np.clip(g / np.percentile(g, 99.5), 0, 1)


def main():
    I = load_image()

    Gx, Gy, G = gradients(I, "sobel")
    show_row([I, scale(np.abs(Gx)), scale(np.abs(Gy)), scale(G)],
             ["Original", "Sobel $|G_x|$", "Sobel $|G_y|$", "Sobel $G=\\sqrt{G_x^2+G_y^2}$"],
             "q3a_sobel_components.png", "Problem 3(a): Sobel gradient components")

    _, _, Gp = gradients(I, "prewitt")
    E = canny(I)
    show_row([I, scale(Gp), scale(G), E.astype(float)],
             ["Original", "Prewitt magnitude", "Sobel magnitude", "Canny edges (default)"],
             "q3b_clean.png", "Problem 3(b): Edge detectors on the original image")

    In = I + rng.normal(0.0, 0.05, I.shape)
    _, _, Gpn = gradients(In, "prewitt")
    _, _, Gn = gradients(In, "sobel")
    En = canny(In)
    show_row([np.clip(In, 0, 1), scale(Gpn), scale(Gn), En.astype(float)],
             ["Noisy ($\\sigma$=0.05)", "Prewitt magnitude", "Sobel magnitude", "Canny edges (default)"],
             "q3d_noisy.png", "Problem 3(d): Edge detectors on the noisy image")

    flat = (slice(20, 120), slice(20, 120))
    edges = G > 0.5 * G.max()
    print("Flat-region mean gradient / mean strong-edge gradient (lower = cleaner)")
    for name, gc, gn in [("Prewitt", Gp, Gpn), ("Sobel", G, Gn)]:
        print(f" {name:7s}: clean={gc[flat].mean() / gc[edges].mean():.4f}, "
              f"noisy={gn[flat].mean() / gn[edges].mean():.4f}")
    print(f"Canny edge pixels: clean={E.sum()}, noisy={En.sum()}, "
          f"in flat region clean={E[flat].sum()}, noisy={En[flat].sum()}")
    print(f"Canny: fraction of clean edges preserved in noisy = "
          f"{np.mean(ndimage.binary_dilation(En, iterations=1)[E]):.3f}")


if __name__ == "__main__":
```

```
76    main()
```